

# せいぎょ 制御文

## プログラムの「流れ」をコントロールしよう：制御文（せいぎょぶん）の基本

みなさん、こんにちは！プログラミングの学習は進んでいますか？今日は、JavaScript（ジャバスクリプト）を学ぶ上でとても大切な「制御文（せいぎょぶん）」について解説します。

## I. はじめに：制御文（せいぎょぶん）とは？

ウェブページを作るとき、HTMLは「家の柱（はしら：骨組み）」、CSSは「壁のペンキ」のような見た目を整える役割です。そして、JavaScriptは家に「電気」を通したり、「機械装置（きかいそうち）」を動かしたりする役割を持っています。

そして、**制御文**（Control Statement）は、その機械を動かすための「脳」にあたります。「もしボタンが押されたら、電気をつける」「もしパスワードが違ったら、エラーを表示する」といった判断（はんだん）をプログラムに教えるのが、**制御文**の役割です。

これを使えるようになると、Webサイトがまるで生きているように、ユーザーの動きに合わせて賢く<sup>賢</sup> かしこく<sup>かしこく</sup> 反応するようになります。まずは、最も基本的な判断のルールである「if文」から見ていきましょう。

## II. 「もし～なら」を決める：if文（イフぶん）

プログラミングで一番よく使うのが「**if文**（if statement）」です。これは「もし～なら、これをする」という条件（じょうけん）を作るための命令です。

`if if else else` というキーワードを使って書きます。

```
if ( 条件 1 ) {  
    処理 1  
} else if ( 条件 2 ) {  
    処理 2  
} else {  
    処理 3  
}
```

- 条件 1 が正しいとき、処理 1 が実行されます
- 条件 1 が間違っていて、条件 2 が正しいとき、処理 2 が実行されます
- 条件 1 も条件 2 が間違っているとき、処理 3 が実行されます。

もう少し具体的に、留学生のみなさんにも馴染みのある「雨と傘」の例で考えてみましょう。

- 「もし、雨が降っているなら、傘を持っていきます」
- 「そうでなくて、くもりなら、折りたたみ傘を持っていきます」

- 「それ以外（晴れ）なら、傘<sup>かさ</sup>は持っていきません」

これをJavaScriptのコードのフォーマットを使って書くと、次のようになります。

```
if (雨が降っています) {
  <ruby>傘<rt>かさ</rt></ruby>を持ていきます
} else if (くもりです) {
  折りたたみ<ruby>傘<rt>かさ</rt></ruby>をもっていきます
} else {
  <ruby>傘<rt>かさ</rt></ruby>は持っていきません
}
```

今度は実際のJavaScriptです。テストの評価<sup>れい</sup>を例にして書いてみます。

```
let score = 95; // テストの点数

if (score >= 90) {
  // 条件が true (正しい) のときだけ動 (うご) きます
  console.log("成績は「優」です。素晴らしい!");
} else if (score >= 70) {
  // 90点未満で、かつ70点以上のとき
  console.log("成績は「良」です。よく頑張りました。");
} else {
  // どの条件にも合わなかったとき
  console.log("次はもっと頑張らしましょう!");
}
```

### か かた 書き方のポイント

- {} (波括弧) の使い方：条件が正しいときに動かしたい処理は、必ず {} の中に書きます。
- どれかひとつ (ひとつ) が選ばれる：上から順番にチェックして、最初に条件が合ったブロックだけが動<sup>うご</sup>きます。
- const と let：値を変えない要素などは **const**、点数や回数のように変わる数字は **let** を使うのが現代のプログラミングの良い習慣です。

if文を使うことで、ウェブページはユーザーの入<sup>にゅうりょく</sup>力に対して「かしこく反<sup>はんのう</sup>応する」ことができるようになります。

## III. 正しく比<sup>えんざん</sup>べるための道具：比<sup>えんざん</sup>較演算子と論理演算子

条件を作るときには、「比<sup>えんざん</sup>べるための記号」を使います。これを「比<sup>えんざん</sup>較演算子 (ひかくえんざんし / Comparison operators)」と呼びます。

特に大切なのが === (厳密<sup>げんみつ</sup>に等しい) です。== という記号もありますが、これはデータ型 (数字か文字か) をあいまいに<sup>判定</sup> はんていするため、間違い (バグ) の原因になりやすいです。プロの現場では、必ず === を

使うのがルールです。

主要な演算子<sup>えんざん</sup>をまとめました。

| 記号  | 意味                      | やさしい日本語での例 <sup>れい</sup>                           |
|-----|-------------------------|----------------------------------------------------|
| === | 厳密 <sup>げんみつ</sup> に等しい | score === 100 （点数が100点のとき）                         |
| >   | より大きい                   | score > 80 （点数が80点より高いとき）                          |
| &&  | かつ (AND)                | age >= 18 && hasTicket === true （18歳以上で、かつチケットもある） |
|     | または (OR)                | age < 18    age > 70 （18歳未満、または、70歳以上）             |
| !   | ではない (NOT)              | !isLoggedIn （ログインしていない状態）                          |

## IV. たくさんの選択肢<sup>せんたくし</sup>から選ぶ：switch文（スイッチぶん）

多くの条件をif文だけで書くと、コードが長くなって大変です。そんな時に便利なのが「switch文」です。「ひとつの変数<sup>へんすう</sup>の値」によって、たくさんの選択肢<sup>せんたくし</sup>から処理を分けたいときは、switch文を使いましょう。

```
const dayNum = 1; // 1は月曜日、2は火曜日...

switch (dayNum) {
  case 1:
    console.log("今日は月曜日です。");
    break;
  case 2:
    console.log("今日は火曜日です。");
    break;
  default:
    console.log("月曜と火曜以外の曜日です。");
}
```

### 大切なキーワード

- case：値がこれと一致（いっち）するかをチェックします。
- break：処理が終わったら外に出る命令です。これを忘れると、次のcaseの処理も勝手に動いてしまう（フォールスルー）ので注意してください。
- default：どのcaseにも合わなかったときに動く「その他」の枠です。

if文を何度も重ねるよりも、switch文を使うことで「パッと見てすぐに意味がわかる」ようになります。つまり、「コードの読みやすさ」がグンと上がります。これを「コードの可読性<sup>かどくせい</sup>（かどくせい）が上がる」と言うことがあります。

## V. 決まった回数を繰り返す：for文（フォーぶん）

プログラミングのすごいところは、同じ仕事を自動で何度も繰り返せることです。決まった回数を繰り返すには「for文」を使います。

for文には、動かすために必要な「3つの要素」があります。

1. 初期設定 (しよき せってい) : `let i = 0` (カウンターを0から始める)
2. 継続条件 (けいぞく じょうけん) : `i < 3` (iが3より小さい間は続ける)
3. 増減処理 (ぞうげん しょり) : `i++` (1回終わるごとに、iを1増やす)

#### 動きの流れ (i < 3 の場合)

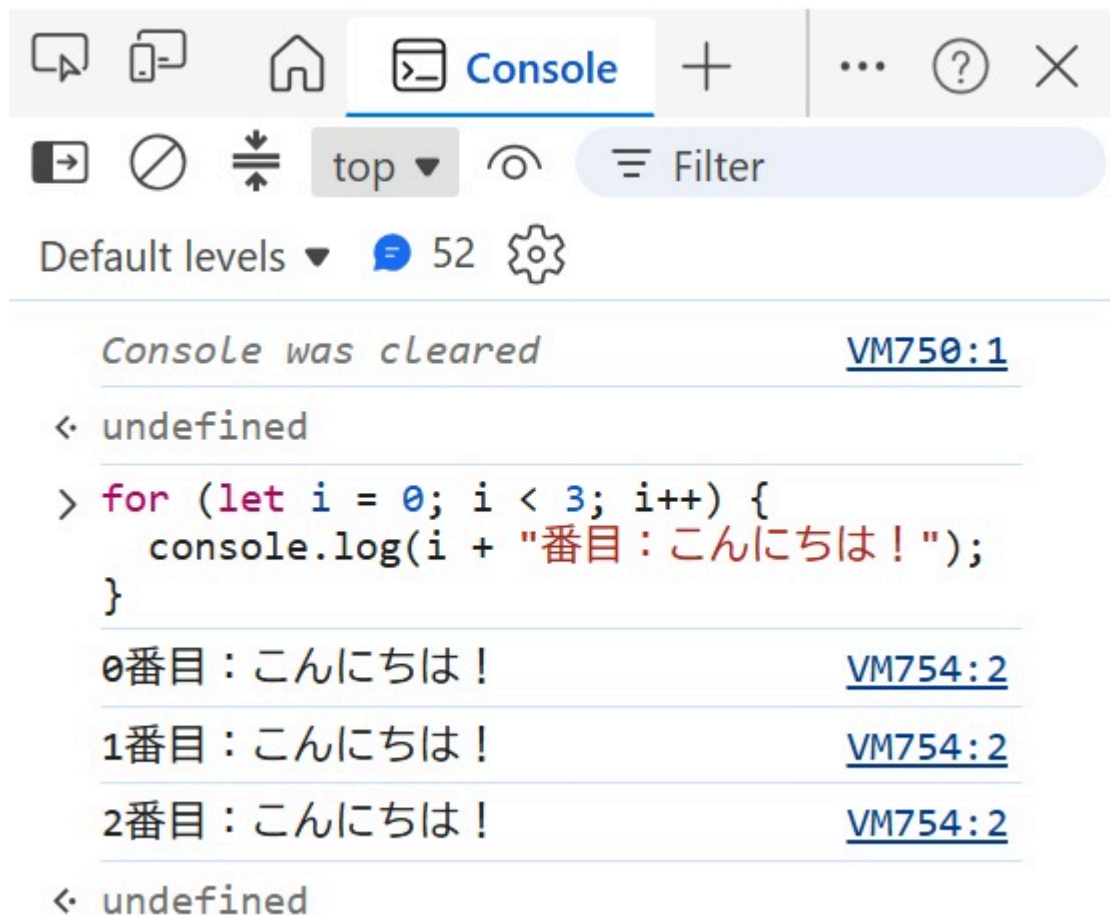
- 1回目 : i は 0。条件 (`0 < 3`) は正しい。処理を実行。→ i が 1 になる。
- 2回目 : i は 1。条件 (`1 < 3`) は正しい。処理を実行。→ i が 2 になる。
- 3回目 : i は 2。条件 (`2 < 3`) は正しい。処理を実行。→ i が 3 になる。
- 終了 : i は 3。条件 (`3 < 3`) は正しくない。ここで止まる。

手作業で100回書く代わりに、for文なら3行で済みます。この効率性 (こうりつせい) があるからこそ、プログラマーは1,000人以上のユーザーデータを一瞬で処理したり、複雑なアニメーションを作ったりすることができるのです。

5回「こんにちは！」と表示するプログラムです。

```
for (let i = 0; i < 3; i++) {
  console.log(i + "番目：こんにちは!");
}
```

実際、どんなふうになるか見てみましょう。F12 キー (ノートPCは Fn + F12) を使ってコンソールを開いてから、上のコードを貼り付けて、Enterキーを押してみてください。



## VI. 条件が合うまで続ける：while文（ワイルぶん）

「何回繰り返すか」は決まっていなくても、「条件が合っている間はずっと続けたい」という時は「while文」が便利です。

```
let count = 1;
while (count <= 3) {
  console.log(count + "回目の処理です");
  count++;
}
```

先ほどと同じように、F12 キー（ノートPCは Fn + F12）を使ってコンソールを開いてから、上のコードを貼り付けて、Enterキーを押してみてください。

```
> let count = 1;
  while (count <= 3) {
    console.log(count + "回目の処理です");
    count++;
  }
```

1回目の処理です [VM758:3](#)

2回目の処理です [VM758:3](#)

3回目の処理です [VM758:3](#)

◀ 3

⚠ 警告：無限ループ（むげんループ）に注意！ while文で一番怖いのは、プログラムが止まらなくなる「無限ループ」です。条件がいつまでも true（正しい）のままだと、ブラウザが固まってしまいます。特に continue（後述）を使うときは注意です。数字を増やす処理（count++ など）の前に continue を書いてしまうと、数字が増えないまま次の回に進んでしまい、永遠にループが終わらなくなります。

## VII. ループを自由にコントロールする：break と continue

繰り返しの途中で、「ここで止めたい」「今回は飛ばしたい」とすることがあります。

- break：繰り返しをその場で「強制終了（きょうせいしゅうりょう）」します。
- continue：今回の回だけ「スキップ（飛ばす）」して、次の回へ進みます。

```
for (let i = 1; i <= 5; i++) {
  if (i === 3) {
    // iが3のときだけ特別な動（うご）きをします
    // continueなら「3」を表示（ひょうじ）せずに、すぐ「4」に進みます
    continue;
  }
  console.log(i);
}
```

先ほどと同じように、F12 キー（ノートPCは Fn + F12）を使ってコンソールを開いてから、上のコードを貼り付けて、Enterキーを押してみてください。

コンソールに実行した結果が表示されます。1 2 4 5 が表示されて、3 だけ表示されません。

```
> for (let i = 1; i <= 5; i++) {
  if (i === 3) {
    // iが3のときだけ特別な動（うご）きを
    // します
    // continueなら「3」を表示（ひょう
    // じ）せずに、すぐ「4」に進みます
    continue;
  }
  console.log(i);
}
```

|   |                         |
|---|-------------------------|
| 1 | <a href="#">VM762:7</a> |
| 2 | <a href="#">VM762:7</a> |
| 4 | <a href="#">VM762:7</a> |
| 5 | <a href="#">VM762:7</a> |

◀ undefined

これらを使いこなすことで、無駄な計算を減らし、プログラムをより効率的<sup>こうりつ</sup>に動かせるようになります。

## VIII. まとめと学習のアドバイス

今日学んだ「制御文」<sup>せいぎょ</sup>は、Webサイトに「知能」<sup>ちのう</sup>を与えるための基礎です。if文で判断し、ループで大量の仕事<sup>しごと</sup>をこなす。これがプログラミングの核（かく）となる部分です。

上達するためのアドバイスです：

1. ブラウザの「コンソール」を使おう：F12（またはFn + F12）キーを押して、コンソールに直接コードを打ち込んでみてください。すぐに結果が見えるので、最高の練習になります。
2. エラーは「ヒント」だと考えよう：赤い文字でエラーが出ても、それは「間違いの場所」を教えてくれるメッセージです。恐れずに読みましょう。
3. 小さなゲームを作ってみよう：ソースコンテキストにある「数当てゲーム」<sup>あ</sup>のように、小さな目標<sup>ちかみち</sup>を作ってコードを書くのが一番の近道です。

制御文<sup>せいぎょ</sup>をマスターすれば、あなたのJavaScriptはもっと自由に、楽しく動かせるようになります。一歩ずつ進んでいきましょう。<sup>いっぽ</sup>

頑張りましょう！